

MAT 70725 – Probabilidad y Estadística

Guía de R para el control de laboratorio

Laboratorios 1, 2 y 3: de cero a modelos de probabilidad

Cómo usar esta guía. Va en orden: cada sección usa solo lo de las anteriores. Lo ideal es tener R o RStudio abierto y *ejecutar cada bloque de código*, mirando qué devuelve. **RStudio se puede descargar desde docs.posit.co/ide/user.** Al final hay 5 ejercicios que combinan los tres labs y sus soluciones completas, y los comandos principales están resumidos en una hoja aparte. Todo el código fue ejecutado en R para verificar los resultados numéricos.

Índice		14 Variable continua: la densidad	9
0. Qué puede preguntar el control	3	15 Esperanza, varianza y F en el caso continuo	10
Parte I – R desde cero (Lab 1)	3	Parte III – Modelos de probabilidad (Lab 3)	11
1 La consola como calculadora	3	16 La familia $d / p / q / r$	11
2 Objetos y asignación	4	17 Modelos discretos	12
3 Vectores: la idea central	4	18 Modelos continuos	13
4 Comparaciones, lógica y selección	4	19 Teórico vs empírico	15
5 Conjuntos, eventos y probabilidad clásica	5	20 Construir un modelo propio: Gumbel	16
6 Conteo	6		
7 Funciones y gráficos	6	Parte IV – Antes del control	17
8 Simulación y reproducibilidad	6	21 Lista de errores a evitar	17
		22 Detalles a tener en cuenta en las guías del curso	17
Parte II – Variables aleatorias (Lab 2)	7	Parte V – Ejercicios	18
9 Construir una variable discreta desde el espacio muestral	7	23 Ejercicios	18
10 Función de cuantía	7	Parte VI – Soluciones	20
11 Probabilidades, esperanza y varianza (discreto)	8	24 Soluciones	20
12 Función de distribución (discreto)	8	A. Respuestas para verificar los ejercicios de los labs	23
13 Simular y comparar	9		

0. Qué puede preguntar el control

Los tres laboratorios trabajan principalmente estos tipos de tarea:

1. **Calcular una probabilidad con conteo:** construir Ω y un evento con vectores, y dividir $\text{length}(A)/\text{length}(\Omega)$.
2. **Simular** un experimento con `sample` o una función `r...`, con `set.seed`, y comparar con el valor teórico.
3. **Construir la cuantía** de una variable discreta con `expand.grid + table`, y sacar E , V y F .
4. **Variable continua:** encontrar la constante, usar `integrate` para probabilidades, esperanza, varianza, y `uniroot` para percentiles.
5. **Modelos:** elegir bien entre d , p , q , r y poner bien los parámetros (sd vs $varianza$, $rate$, geométrica que cuenta fracasos, $<$ vs $\leq$ en discretas).
6. **Gráficos:** `plot` con `type="h"`, `"s"`, `"l"`, agregar curvas con `lines`, e histograma con `hist(..., probability=TRUE)`.

La regla de oro

Casi todos los errores en un control de R no son de programación, son de **traducir** la frase en palabras a la expresión correcta: ¿es $P(X < 5)$ o $P(X \leq 5)$? ¿el parámetro es la desviación o la varianza? ¿la media o la tasa? Antes de escribir código, escribe la probabilidad en notación matemática.

Parte I – R desde cero (Laboratorio 1)

1 La consola como calculadora

R lee una instrucción, la evalúa y muestra el resultado. Lo que va después de `#` es comentario y se ignora.

```
2 + 3      # 5
10 / 4     # 2.5 (division real, no entera)
2^5        # 32 (potencia: ^, no ** como en Python)
sqrt(16)   # 4
exp(2)     # e^2 = 7.389...
log(8)     # logaritmo NATURAL (ln)
log10(8)   # base 10
log(8, base = 2) # 3
```

Trampa típica

`log()` en R es el logaritmo natural, no en base 10. Esto importa en la log-normal (Parte III).

Descargar RStudio

RStudio se puede descargar desde:

<https://docs.posit.co/ide/user/#rstudio-ide-oss-downloads>

Todo el código de esta guía usa R base, así que funciona igual en RStudio o en RGui.

Ayuda: `?sqrt` abre la documentación. `Ctrl + L` limpia la consola pero *no* borra los objetos; para borrar todo se usa `rm(list = ls())`.

2 Objetos y asignación

Se guarda un valor en un objeto con `<-`. Escribir el nombre solo lo muestra; envolver en paréntesis asigna y muestra a la vez.

```
x <- 5
y <- 3
x + y      # 8
x^y        # 125
(z <- 10)  # asigna 10 a z y lo imprime
```

	Python	R
Asignar	<code>x = 5</code>	<code>x <- 5</code>
Comparar igualdad	<code>==</code>	<code>==</code>
Potencia	<code>**</code>	<code>^</code>
Resto de división	<code>%</code>	<code>%%</code>
Primer elemento	<code>x[0]</code>	<code>x[1]</code>
Y / O / NO lógicos	<code>and or not</code>	<code>& !</code>

3 Vectores: la idea central

En R casi todo es un vector, y las operaciones se aplican **elemento a elemento sin ciclos**. Esto es lo más importante de todo el laboratorio: en vez de un `for`, se opera sobre el vector completo.

```
x <- c(11, 7, 5, 8, 2) # c() = "combinar"
x[1]                  # 11 <- la indexacion empieza en 1
x[2:4]                # 7 5 8
1:10                  # 1 2 3 ... 10
seq(0, 1, by = 0.1)   # 0.0 0.1 ... 1.0
rep(3, 5)             # 3 3 3 3 3

x + 2                 # 13 9 7 10 4 (suma 2 a cada uno)
x^2                   # cada elemento al cuadrado
length(x)
sum(x)
mean(x)
min(x)
max(x)
sort(x)
```

Por qué esto importa para probabilidad

Un espacio muestral es un vector de resultados. Una simulación es un vector de miles de resultados. Una cuantía es un vector de probabilidades. Todo el curso consiste en crear vectores y resumirlos con `sum`, `mean`, `length`.

4 Comparaciones, lógica y selección

Una comparación aplicada a un vector devuelve un vector de TRUE/FALSE. Ese vector lógico sirve para dos cosas:

```
x <- 1:10 # OJO: reemplaza el x anterior; ejecutar esta línea primero
```

```
x > 5          # FALSE FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE
x %% 2 == 0    # TRUE donde x es par (resto 0 al dividir por 2)

# 1) SELECCIONAR: quedarse con los elementos que cumplen
x[x > 5]       # 6 7 8 9 10
x[x > 3 & x < 8] # 4 5 6 7 (& = y)
x[x %% 2 == 0 | x == 5] # 2 4 5 6 8 10 (/ = o)

# 2) CONTAR: TRUE vale 1 y FALSE vale 0
sum(x > 5)     # 5 -> cuantos cumplen
sum(x > 5) / length(x) # 0.5 -> que proporción cumple
```

Los operadores lógicos: &, | y !

& significa "y" TRUE solo si se cumplen *las dos* condiciones
 | (la barra vertical) significa "o" TRUE si se cumple *al menos una*
 ! significa "no" invierte: TRUE pasa a FALSE y viceversa

Ejemplo, leyendo `x[x %% 2 == 0 | x == 5]` de adentro hacia afuera:

x:	1	2	3	4	5	6	7	8	9	10	
x %% 2 == 0	F	T	F	T	F	T	F	T	F	T	# es par?
x == 5	F	F	F	F	T	F	F	F	F	F	# es 5?
par 5	F	T	F	T	T	T	F	T	F	T	# par o 5
x[...]	-> 2 4 5 6 8 10										# los que dieron TRUE

El truco más útil del control

`sum(condicion) / n` = proporción de elementos que cumplen la condición. `sum` cuenta los TRUE (cada uno vale 1) y al dividir por el total queda una proporción. Si el vector es una simulación, eso es una **frecuencia relativa**, o sea una estimación de la probabilidad.

5 Conjuntos, eventos y probabilidad clásica

Un dado: $\Omega = \{1, \dots, 6\}$. Un evento es un subconjunto, que se construye seleccionando.

```
Omega <- 1:6
A <- Omega[Omega %% 2 == 0] # par: 2 4 6
B <- Omega[Omega < 4]      # 1 2 3

intersect(A, B)           # A inter B = 2
union(A, B)               # A union B = 2 4 6 1 3
setdiff(Omega, A)         # complemento de A = 1 3 5

# Espacio finito equiprobable: P(A) = #A / #Omega
length(A) / length(Omega) # 0.5
length(intersect(A, B)) / length(Omega) # 1/6
length(union(A, B)) / length(Omega)     # 5/6
```

Pruébalo

Comprueba numéricamente la regla de la suma: $P(A) + P(B) - P(A \cap B)$ debe dar lo mismo que $P(A \cup B)$.

6 Conteo

```
factorial(5)      # 120
choose(12, 3)     # coeficiente binomial  $C(12,3) = 220$ 
combn(1:5, 3)     # matriz: cada COLUMNA es una combinacion
ncol(combn(1:5, 3)) # 10 = choose(5,3)
```

choose da el número; combn construye las combinaciones explícitamente (útil para verificar o para contar las que cumplen algo).

7 Funciones y gráficos

```
f <- function(x){
  x^2 + 2*x - 1    # lo ultimo evaluado es lo que se devuelve
}
f(2)              # 7
f(1:5)            # se aplica a cada elemento: 2 7 14 23 34
```

```
x <- seq(-3, 3, by = 0.01)
plot(x, x^2, type = "l", xlab = "x", ylab = "f(x)", main = "f(x) = x^2")
```

El argumento type cambia todo el gráfico, y en el curso aparecen cuatro:

type	Uso en el curso
"p"	puntos (por defecto)
"l"	línea continua: densidades y F continuas
"h"	barras verticales: cuantía discreta
"s"	escalera: F de una discreta

Para agregar sobre un gráfico ya dibujado: lines() (curvas) y points() (puntos). lines no crea un gráfico nuevo, así que siempre va después de un plot o hist.

8 Simulación y reproducibilidad

sample(v, size = n, replace = TRUE) saca elementos al azar de v. Para lanzamientos independientes de un dado hay que usar replace = TRUE (el dado no "pierde" caras).

```
set.seed(70725)                # fija la semilla
n <- 10000
x <- sample(1:6, size = n, replace = TRUE)
sum(x %% 2 == 0) / n           # aprox. 0.5 = P(par)
```

Por qué la simulación funciona

Es la **ley de los grandes números**: la frecuencia relativa de un evento converge a su probabilidad cuando n crece. El error típico es del orden de $\sqrt{p(1-p)/n}$; con $n = 10000$ y $p = 0.5$ eso es ≈ 0.005 . Por eso con 10 000 repeticiones se esperan unos dos decimales correctos, no más. Con $n = 100$ el error es ≈ 0.05 : si en el control piden comparar $n = 100, 1000, 10000$, la conclusión es que la estimación se acerca a la teórica y su variación se reduce como $1/\sqrt{n}$.

`set.seed` hace que el "azar" sea repetible: si se ejecuta desde `set.seed(70725)` se obtiene exactamente la misma secuencia. **Ojo:** hay que volver a ejecutar la línea del `set.seed` cada vez; si no, se sigue desde donde quedó el generador y los números cambian.

Desafío del Lab 1, resuelto

Dos dados, $P(\text{suma} \geq 9)$. Exacto: los pares con suma 9, 10, 11, 12 son $4 + 3 + 2 + 1 = 10$, así que $P = 10/36 \approx 0.2778$.

```
10 / 36          # exacto, por equiprobabilidad: 0.2778

set.seed(70725)                                # simulado (codigo del Lab 1)
n <- 10000
dado1 <- sample(1:6, size = n, replace = TRUE)
dado2 <- sample(1:6, size = n, replace = TRUE)
sum(dado1 + dado2 >= 9) / n                    # aprox. 0.28
```

Parte II – Variables aleatorias (Laboratorio 2)

9 Construir una variable discreta desde el espacio muestral

Una variable aleatoria es una **función** que a cada resultado de Ω le asigna un número. En R: se construye Ω como tabla y se calcula X fila por fila (de forma vectorial).

```
Omega <- expand.grid(dado1 = 1:6, dado2 = 1:6) # todas las parejas
nrow(Omega)                                # 36
X <- Omega$dado1 + Omega$dado2              # $ extrae una columna; suma fila a fila
sort(unique(X))                             # recorrido: 2 3 ... 12
```

`expand.grid` genera *todas* las combinaciones de los vectores que recibe; con tres argumentos sirve para tres dados o tres monedas. Para tres monedas, por ejemplo: `expand.grid(m1=0:1, m2=0:1, m3=0:1)` (1 = cara) y $X = \text{Omega}\$m1 + \text{Omega}\$m2 + \text{Omega}\$m3$.

Otras variables útiles

Con las mismas operaciones vectoriales del Lab 1 se pueden definir otras variables: el producto `Omega$dado1 * Omega$dado2`, la diferencia `Omega$dado1 - Omega$dado2`, o un evento como "salen dobles", `Omega$dado1 == Omega$dado2`, cuya probabilidad es `sum(...)/nrow(Omega)`. Ojo: `max` y `min` devuelven *un solo número* para todo el vector, así que no sirven para "el mayor de los dos dados" fila por fila.

10 Función de cuantía

```
conteo <- table(X)                            # cuantas veces aparece cada valor
fx <- conteo / nrow(Omega)                     # divide por 36 -> probabilidades
sum(fx)                                         # 1 (chequeo obligatorio)

x <- as.numeric(names(fx))                    # los VALORES (estaban como nombres)
p <- as.numeric(fx)                           # las PROBABILIDADES
```

Trampa típica

table guarda los valores de X como *nombres* (texto), no como números. Por eso hace falta `as.numeric(names(fx))`. Si no se separa en x y p , operaciones como `sum(x*p)` fallan o dan basura.

11 Probabilidades, esperanza y varianza (discreto)

Con x y p separados, una probabilidad es sumar las p de los x que cumplen:

```
sum(p[x >= 9])      # P(X >= 9) = 10/36
sum(p[x < 5])       # P(X < 5)  = 6/36
sum(p[x >= 5 & x <= 8]) # P(5 <= X <= 8)

EX <- sum(x * p)     # E(X) = 7
EX2 <- sum(x^2 * p)  # E(X^2)
VX <- EX2 - EX^2      # V(X) = 35/6 = 5.833
sqrt(VX)             # desviacion estandar
```

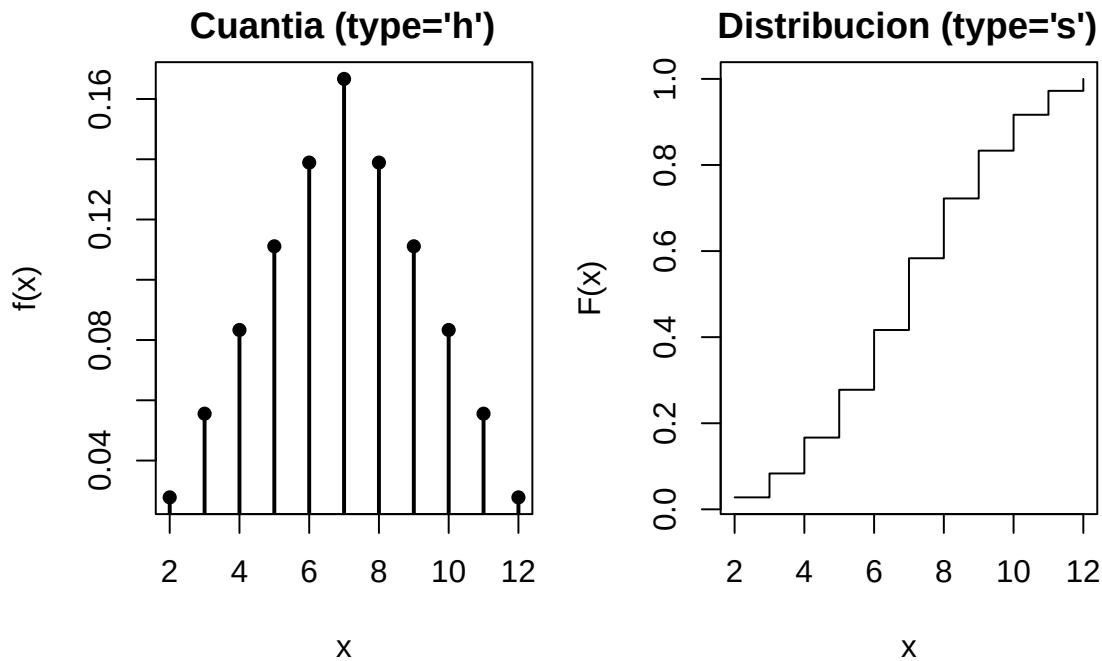
Esperanza y varianza como mecánica

Imagina masas p_i colocadas en las posiciones x_i de una regla. $E(X) = \sum x_i p_i$ es el **centro de masa**: el punto donde la regla se equilibra (para los dos dados, en 7, por simetría). $V(X) = \sum (x_i - \mu)^2 p_i$ es el **momento de inercia** respecto al centro: mide qué tan lejos está repartida la masa. La fórmula $V = E(X^2) - E(X)^2$ es el teorema de Steiner (ejes paralelos).

12 Función de distribución (discreto)

$F(x) = P(X \leq x)$ acumula las probabilidades: `cumsum` hace exactamente eso.

```
Fx <- cumsum(p)
data.frame(x, p, Fx)      # tabla ordenada x | p | F(x)
plot(x, Fx, type = "s", xlab = "x", ylab = "F(x)") # escalera
plot(x, p, type = "h", lwd = 2)                   # cuantia
```

La escalera salta justo en cada valor posible, y el tamaño del salto es $P(X = x)$. Entre valores es plana porque ahí no hay probabilidad.

13 Simular y comparar

```
set.seed(70725)
n <- 10000
Xsim <- sample(1:6, size = n, replace = TRUE) + sample(1:6, size = n, replace = TRUE)
prop.table(table(Xsim)) # frecuencias relativas (comparar con fx)
mean(Xsim)
var(Xsim) # aprox. 7 y aprox. 5.83
```

`prop.table(table(v))` es lo mismo que `table(v)/length(v)`.

14 Variable continua: la densidad

Para $f_Y(y) = ky$, $0 < y < 2$: el área total debe ser 1, $\int_0^2 ky dy = 2k = 1 \Rightarrow k = 1/2$. **La constante se saca a mano** y luego se verifica en R.

```
f <- function(y){ 0.5 * y }
integrate(f, lower = 0, upper = 2) # 1 with absolute error < 1.1e-14
integrate(f, lower = 0.5, upper = 1.5) # P(0.5 < Y < 1.5) = 0.5
integrate(f, lower = 0.5, upper = 1.5)$value # solo el numero: 0.5
```

Trampa típica

`integrate` devuelve un objeto que *imprime* un número con texto, pero no es un número. Para operar con él (restar, elevar al cuadrado, comparar) hay que extraer `$value`. Olvidarlo es el error más común del Lab 2.

Densidad = masa por unidad de largo

$f(y)$ no es una probabilidad; es como la densidad lineal de una barra. La probabilidad es la *masa* de un tramo: el área bajo la curva. Por eso $P(Y = 1) = 0$ (un punto no tiene largo) y por eso f puede valer más que 1 sin problema.

15 Esperanza, varianza y F en el caso continuo

Las sumas se vuelven integrales. Como `integrate` necesita una *función*, se definen funciones auxiliares:

```
g1 <- function(y){ y * f(y) }
g2 <- function(y){ y^2 * f(y) }
EY <- integrate(g1, lower = 0, upper = 2)$value # 4/3
EY2 <- integrate(g2, lower = 0, upper = 2)$value # 2
VY <- EY2 - EY^2 # 2/9 = 0.2222
```

$F(y) = \int_0^y f(t) dt$. El límite superior es el argumento:

```
F <- function(y){ integrate(f, lower = 0, upper = y)$value }
F(1) # 0.25 (exacto: y^2/4)

y <- seq(0, 2, by = 0.01)
Fy <- sapply(y, F) # NO sirve F(y) con un vector -> sapply
plot(y, Fy, type = "l")
```

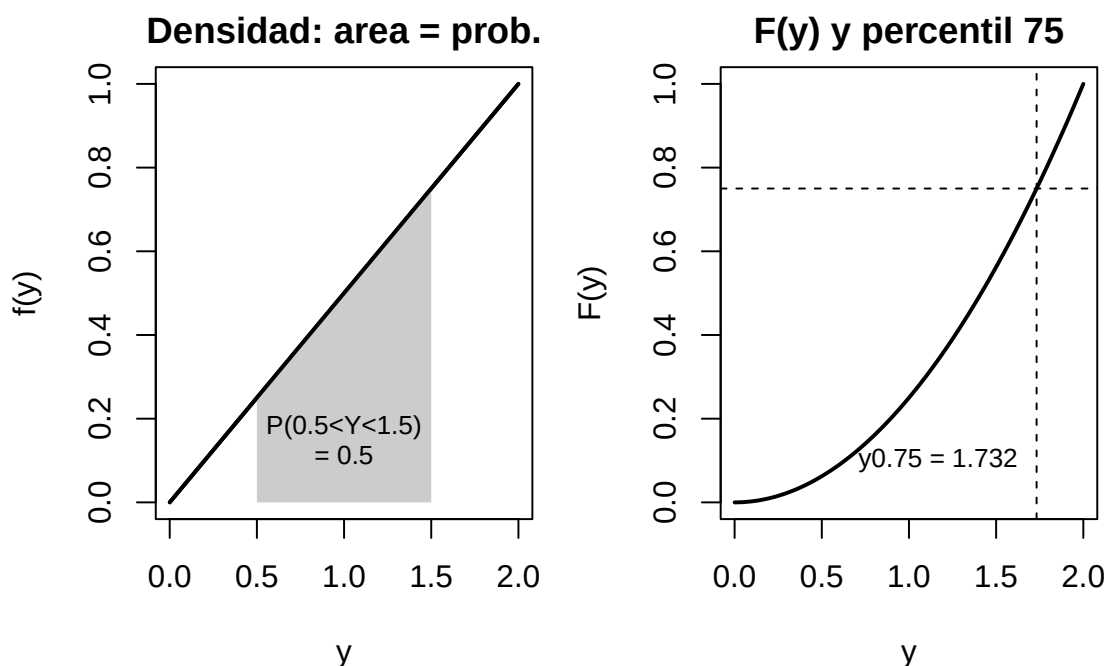
Por qué `sapply`

A diferencia de casi todo en R, `integrate` **no es vectorial en sus límites**: `F(c(0.5, 1))` da error. `sapply(y, F)` aplica `F` a cada elemento de `y` y por separado y junta los resultados en un vector. Es el equivalente a `[F(t) for t in y]` de Python.

Percentiles con `uniroot`

El percentil q es el y_q tal que $F(y_q) = q$. Eso es encontrar la raíz de $F(y) - q$:

```
y75 <- uniroot(function(y){ F(y) - 0.75 }, interval = c(0, 2))$root
y75 # 1.732 (exacto: sqrt(3))
```



Errores con uniroot

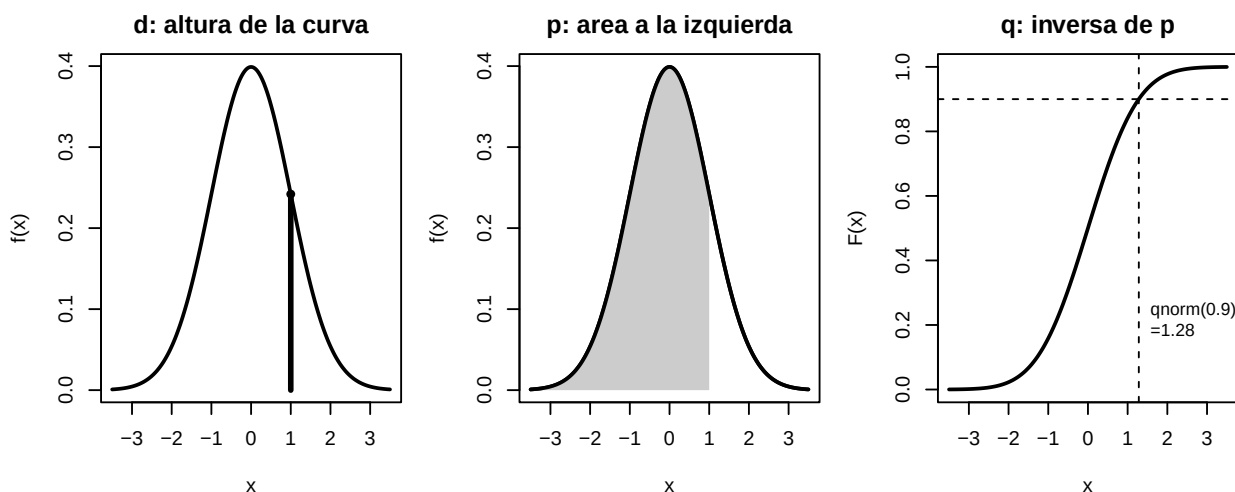
- El intervalo debe ser el soporte (o estar dentro), y la función debe cambiar de signo entre sus extremos; si no, da error *"values at end points not of opposite sign"*.
- También se extrae con `$root`, igual que `$value`.
- Se puede comprobar: $F(y_{75})$ debe dar ≈ 0.75 .

Parte III – Modelos de probabilidad (Laboratorio 3)

16 La familia d / p / q / r

R ya trae los modelos. El nombre es **prefijo + modelo**:

Prefijo	Qué calcula	Pregunta que responde	Ejemplo
d	cuantía / densidad	$P(X = x)$ (solo discretas)	<code>dbinom(5, size = 20, prob = 0.3)</code>
p	distribución F	$P(X \leq x)$	<code>pnorm(75, mean = 70, sd = 8)</code>
q	cuantil (inversa de F)	¿qué x deja prob. q abajo?	<code>qnorm(0.95, mean = 70, sd = 8)</code>
r	simulación	genera n observaciones	<code>rnorm(10000, mean = 70, sd = 8)</code>



Cómo traducir una pregunta

"probabilidad de que sea exactamente k "	d (solo discretas)
"a lo más / menor o igual / como máximo"	p
"más de / mayor que"	$1 - p(\dots)$ o $p(\dots, \text{lower.tail} = \text{FALSE})$
"percentil, cuantil, valor que no es superado con prob."	q
"genere / simule"	r

17 Modelos discretos

17.1 Binomial: $X \sim \text{Bin}(n, p)$

Número de éxitos en n ensayos independientes con probabilidad p . $E = np$, $V = np(1 - p)$. En R: `size = n`, `prob = p`.

```
dbinom(5, size = 20, prob = 0.3) # P(X = 5)
pbinom(5, size = 20, prob = 0.3) # P(X <= 5)
sum(dbinom(0:5, size = 20, prob = 0.3)) # lo mismo, sumando la cuantia
```

La trampa más importante: discretas y desigualdades

En una discreta, $<$ y $\leq$ **no son lo mismo**, porque los puntos sí tienen probabilidad. Con X entero:

$$P(X < b) = P(X \leq b - 1), \quad P(X \geq a) = 1 - P(X \leq a - 1), \quad P(X > a) = 1 - P(X \leq a).$$

```
pbinom(8, size = 20, prob = 0.3) - pbinom(3, size = 20, prob = 0.3) # P(4 <= X <= 8)
pbinom(7, size = 20, prob = 0.3) - pbinom(4, size = 20, prob = 0.3) # P(4 < X < 8) = P(5 <= X <= 7)
1 - pbinom(14, size = 30, prob = 0.4) # P(X >= 15) ; 14, no 15!
```

El Lab 3 escribe $P(4 < X < 8)$ como $F(8) - F(4) - P(X = 8)$, que es lo mismo que $F(7) - F(4)$. Recomendación: siempre **reescribir con $\leq$ y enteros** y usar la forma $F(b') - F(a')$; es más difícil equivocarse. Y en caso de duda, `sum(dbinom(a:b, size = n, prob = p))` con los enteros exactos que cuentan nunca falla.

En continuas da igual: $P(a < X < b) = P(a \leq X \leq b) = F(b) - F(a)$.

Cuantiles discretos. `qbinom(0.90, size = 20, prob = 0.3)` devuelve el *menor* x entero tal que $P(X \leq x) \geq 0.90$. Es exactamente lo que piden preguntas tipo "menor k tal que $P(X \leq k) \geq 0.95$ " (`qpois(0.95, lambda = 6)`). Se puede verificar: `pbinom(qbinom(0.9, size = 20, prob = 0.3), size = 20, prob = 0.3)` debe ser ≥ 0.9 y el del valor anterior, < 0.9 .

Gráfico de cuantía: `plot(0:20, dbinom(0:20, size = 20, prob = 0.3), type="h", lwd=2)` y luego `points(..., pch=16)`. Al subir `prob` la masa se corre a la derecha; con $p = 0.5$ queda simétrica; al subir `size` se ensancha y se parece a una normal.

17.2 Geométrica (¡ojo con la definición de R!)

Trampa típica

En R, `dgeom`, `pgeom`, etc. modelan $X = \text{número de fracasos antes del primer éxito}$, con recorrido $\{0, 1, 2, \dots\}$ y $E(X) = (1-p)/p$. Si el problema dice "número de *intentos* hasta el primer éxito" ($N = X + 1$, recorrido $\{1, 2, \dots\}$, $E(N) = 1/p$), hay que traducir:

$$P(N = k) = \text{dgeom}(k-1, p), \quad P(N \leq k) = \text{pgeom}(k-1, p), \quad N_{\text{sim}} = \text{rgeom}(n, p) + 1.$$

17.3 Poisson: $Y \sim \text{Poi}(\lambda)$

Número de eventos en un intervalo, con tasa promedio λ . $E = V = \lambda$ (la firma de la Poisson: media y varianza iguales).

```
dpois(3, lambda = 4.5)      # P(Y = 3)
ppois(6, lambda = 4.5)      # P(Y <= 6)
qpois(0.95, lambda = 4.5)   # menor k con P(Y<=k) >= 0.95
set.seed(70725)
Y <- rpois(10000, lambda = 4.5)
mean(Y)
var(Y)      # ambos aprox. 4.5
```

Intuición física

λ escala con el tamaño del intervalo: si llegan 3.2 clientes por hora, en 2 horas $\lambda = 6.4$. Es la misma idea que un flujo constante: duplicar el tiempo duplica el número esperado de llegadas.

18 Modelos continuos

Trampa típica

En continuas da la **altura de la densidad**, *no* una probabilidad. `dunif(4, min = 2, max = 8)` $= 1/6$ es $f(4)$, y $P(X = 4) = 0$. Las probabilidades continuas siempre salen de `p`.

18.1 Uniforme $U(a, b)$

`dunif(x, min=a, max=b)`, `punif`, `qunif`, `runif`. $E = (a+b)/2$, $V = (b-a)^2/12$. `runif(n)` sin más argumentos es $U(0, 1)$: la base de toda simulación (ver Gumbel).

18.2 Exponencial

Tiempo hasta un evento con tasa λ . **R usa** $\text{rate} = \lambda$, y $E = 1/\lambda$, $V = 1/\lambda^2$.

```
pexp(3, rate = 0.5)           # P(X <= 3)
1 - pexp(3, rate = 0.5)       # P(X > 3)
pexp(3, rate = 0.5, lower.tail = FALSE) # lo mismo
qexp(0.90, rate = 0.5)        # percentil 90
```

Trampa típica

Si el enunciado dice "duración *media* de 400 horas", entonces $\text{rate} = 1/400$, no 400. Al aumentar rate la densidad cae más rápido (eventos más frecuentes, tiempos más cortos).

Falta de memoria

La exponencial es como un núcleo radiactivo: no "envejece". $P(X > s + t \mid X > s) = P(X > t)$. Si una ampolla exponencial ya duró 200 h, la probabilidad de que dure 500 h más es la misma que si fuera nueva. En R: $(1 - \text{pexp}(700, \text{rate} = r)) / (1 - \text{pexp}(200, \text{rate} = r))$ coincide con $1 - \text{pexp}(500, \text{rate} = r)$.

18.3 Normal $N(\mu, \sigma^2)$

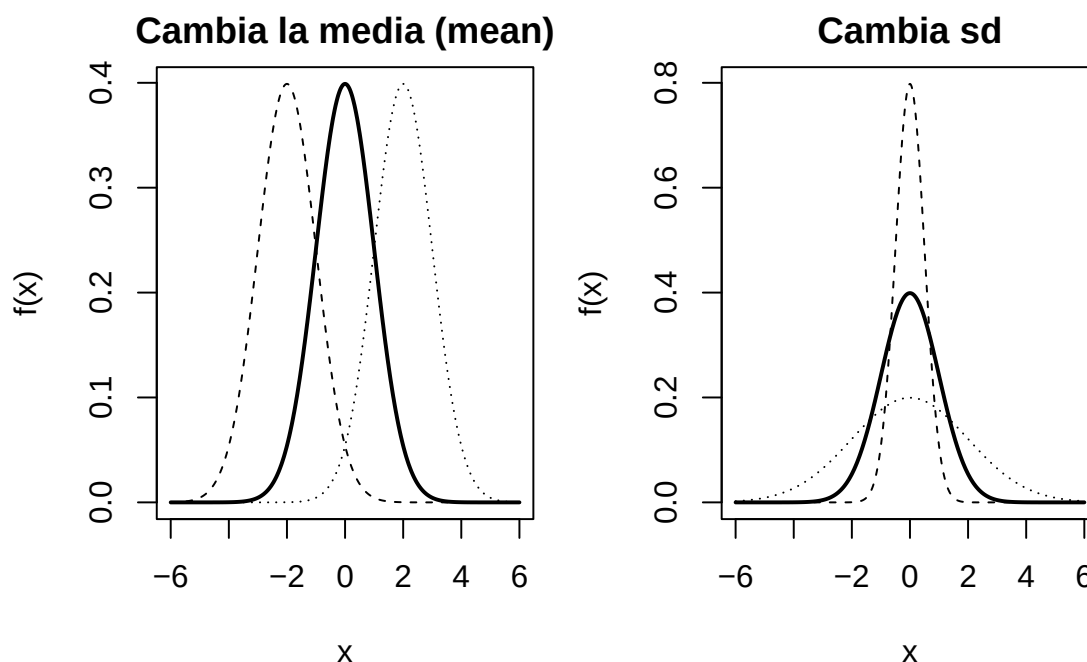
Error clásico

La notación $N(70, 8^2)$ da la **varianza** 64, pero R pide la **desviación estándar**: $\text{sd} = 8$. Si el enunciado dice $N(100, 225)$, entonces $\text{sd} = 15$, no 225.

```
pnorm(75, mean = 70, sd = 8)           # P(X <= 75)
pnorm(75, mean = 70, sd = 8, lower.tail = FALSE) # P(X > 75)
pnorm(80, mean = 70, sd = 8) - pnorm(60, mean = 70, sd = 8) # P(60 < X < 80)
qnorm(0.95, mean = 70, sd = 8)          # percentil 95
# intervalo central del 95% (colas iguales de 2.5% cada una):
qnorm(0.025, mean = 70, sd = 8)         # a
qnorm(0.975, mean = 70, sd = 8)         # b
```

Intervalo con colas iguales

" a y b tales que $P(a < X < b) = 0.95$ con colas iguales" significa que queda $0.05/2 = 0.025$ en cada cola: $a = \text{qnorm}(0.025, \dots)$, $b = \text{qnorm}(0.975, \dots)$. Para cualquier modelo continuo es la misma idea con su q .



La media **traslada** la curva sin cambiar su forma; la desviación la **ensancha** (y la baja, porque el área debe seguir siendo 1).

18.4 Log-normal

X es log-normal si $\log X \sim N(\mu, \sigma^2)$. En R: $\text{meanlog} = \mu$ y $\text{sdlog} = \sigma$ son **la media y desviación de $\log X$** , no de X .

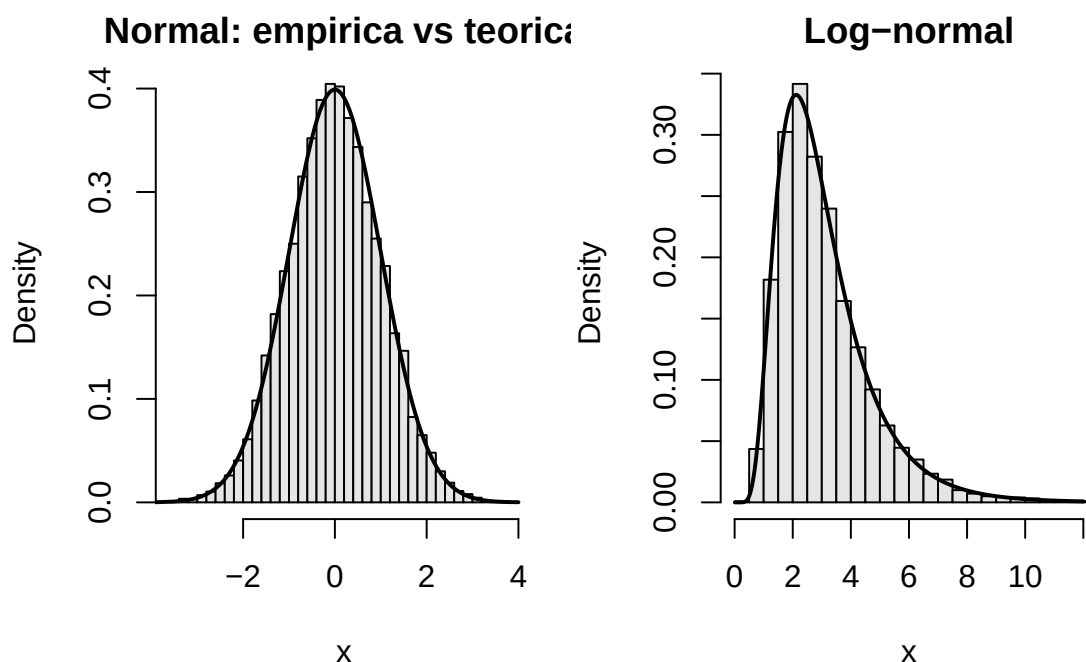
```
plnorm(3, meanlog = 1, sdlog = 0.5)      # P(X <= 3)
qlnorm(0.90, meanlog = 1, sdlog = 0.5)   # percentil 90
set.seed(70725)
X <- rlnorm(10000, meanlog = 1, sdlog = 0.5)
mean(log(X))
sd(log(X))    # aprox. 1 y aprox. 0.5 <- verifica los parametros
mean(X)       # aprox. exp(1 + 0.5^2/2), NO exp(1)
```

Datos útiles: mediana = e^μ (porque log es creciente y la mediana de la normal es μ), $E(X) = e^{\mu + \sigma^2/2}$. Al subir sdlog la distribución se vuelve más asimétrica con cola derecha larga.

19 Teórico vs empírico

Simular y superponer la densidad teórica sobre un histograma *en escala de densidad*:

```
set.seed(70725)
X <- rnorm(10000)
hist(X, probability = TRUE, breaks = 40)  # probability=TRUE es CLAVE
x <- seq(-4, 4, by = 0.01)
lines(x, dnorm(x), lwd = 2)
```



Trampa típica

Sin `probability = TRUE`, el histograma muestra *conteos* (miles) y la densidad (valores menores a 1) queda aplastada en el eje: parece que la curva no aparece. Para discretas, la comparación análoga es `prop.table(table(Xsim))` contra `d...`

Qué decir en un análisis: la media y varianza muestrales se acercan a las teóricas, la forma del histograma sigue a la curva, y las diferencias son fluctuaciones aleatorias que se achican al aumentar n .

20 Construir un modelo propio: Gumbel

Solo para estudio personal

En el Lab 3 esta sección aparece bajo el título "*Para estudio personal: Modelo Gumbel*", al final de la guía y después del desafío. Es material complementario: se incluye aquí por completitud, pero no forma parte del contenido principal de la sesión.

R base no trae la Gumbel, así que se programan sus cuatro funciones a partir de F :

$$F(x) = \exp\left\{-e^{-(x-\alpha)/\beta}\right\}, \quad q(p) = \alpha - \beta \log(-\log p).$$

```
dgumbel <- function(x, alpha, beta){
  z <- (x - alpha) / beta
  (1 / beta) * exp(-z) * exp(-exp(-z))
}
pgumbel <- function(x, alpha, beta){
  z <- (x - alpha) / beta
  exp(-exp(-z))
}
qgumbel <- function(p, alpha, beta){
  alpha - beta * log(-log(p))
}
rgumbel <- function(n, alpha, beta){
  U <- runif(n)
}
```



```
qgumbel(U, alpha, beta)
}
```

Método de la transformada inversa

Si $U \sim U(0, 1)$, entonces $q(U)$ tiene distribución F . Es decir: **para simular cualquier modelo basta su función cuantil y runif**. Intuición: U elige "un percentil al azar" uniformemente, y q lo convierte en el valor de x correspondiente; donde la densidad es alta, F sube rápido y muchos percentiles caen en poco espacio.

Para verificar la simulación: $E(X) = \alpha + \gamma\beta$ con $\gamma \approx 0.5772$ y $V(X) = \pi^2\beta^2/6$. α traslada la curva, β la ensancha. Es el modelo de *máximos* (crecidas de ríos, vientos extremos), por eso tiene cola derecha más larga.

Parte IV – Antes del control

21 Lista de errores a evitar

1. **Indexar desde 0**. En R, `x[1]` es el primero.
2. **Olvidar `$value / $root`** al usar el resultado de `integrate` o `uniroot`.
3. **Usar `F(vector)`** cuando `F` usa `integrate`: usar `sapply`.
4. **`< vs ≤` en discretas**: $P(X \geq 15) = 1 - \text{pbinom}(14, \dots)$.
5. **Varianza en vez de `sd`** en `pnorm/qnorm/rnorm`.
6. **Media en vez de `rate`** en la exponencial.
7. **Geométrica**: R cuenta fracasos, no intentos.
8. `replace = TRUE` en `sample` para lanzamientos repetidos.
9. `probability = TRUE` en `hist` si se va a superponer una densidad.
10. **`max/min` sobre un vector** devuelven un solo número, no un valor por fila.
11. **No volver a ejecutar `set.seed`** antes de repetir una simulación.
12. **Usar `d` como probabilidad** en continuas.
13. **No verificar**: `sum(p)` debe ser 1, `integrate` sobre el soporte debe dar 1, `F(percentil)` debe dar el nivel pedido.

22 Detalles a tener en cuenta en las guías del curso

Las tres guías no tienen errores de cálculo, pero sí algunos puntos que pueden confundir:

- **Lab 2 y Lab 3 usan `F` como nombre de función**. En R, `F` es también una abreviación de `FALSE`. Redefinirlo está permitido y funciona, pero si después se escribe algo como `replace = F` pensando en `FALSE`, se pasa la función y falla. Es más seguro llamarla `Fy` o `FY`, o escribir siempre `TRUE/FALSE` completos.
- **Lab 2, sección 6**: `f <- function(y){ 0.5 * y }` solo vale dentro del soporte $(0, 2)$; fuera de él devuelve valores que no son la densidad (que es 0). Mientras se integre entre 0 y 2 no importa, pero

si en el control se integra en un intervalo que se sale del soporte, hay que cortar los límites al soporte. Una versión segura, usando que TRUE vale 1 y FALSE vale 0: `f <- function(y){ 0.5 * y * (y > 0 & y < 2) }`.

- **Lab 3, sección 2:** la fórmula $P(a < X < b) = F(b) - F(a) - P(X = b)$ es correcta, pero equivale a $F(b - 1) - F(a)$ para enteros, que es más directa.
- **Lab 3, ejercicio d:** en el PDF aparece " $N(100, 152)$ "; es $N(100, 15^2)$ (el exponente se perdió al extraer el texto), o sea $sd = 15$.
- **Las guías reutilizan** X , x , f con significados distintos en cada sección. Si se ejecuta el script desordenado, un objeto puede tener el valor de otra sección. En el control conviene usar nombres propios por ejercicio o ejecutar en orden.

Parte V – Ejercicios

23 Ejercicios

Son cinco ejercicios, y cada uno mezcla varias herramientas de los tres labs. Conviene intentar cada uno en R antes de mirar la solución. En todas las simulaciones se usa `set.seed(70725)`.

Ejercicio 1 (conteo y cuantía). Una urna tiene 5 bolitas rojas (numeradas 1 a 5) y 7 azules (numeradas 6 a 12). Se extraen 4 sin reposición, y sea X el número de rojas.

- Calcule $P(X = 2)$ con `choose`.
- Verifique con `combn(1:12, 4)` y `ncol` que el número total de extracciones posibles es `choose(12, 4)`.
- Obtenga la cuantía completa de X evaluando la fórmula de conteo en el vector `x <- 0:4`, y verifique que suma 1.
- Calcule $E(X)$ y $P(X \geq 1)$.

Ejercicio 2 (variable discreta desde Ω y simulación). Se lanzan dos dados y sea Y el **producto** de los dos resultados.

- Construya la cuantía de Y con `expand.grid`.
- Determine el recorrido de Y y calcule $E(Y)$, $V(Y)$ y $P(Y \leq 6)$.
- Construya F_Y y gráfiquela como escalera.
- Simule 10 000 lanzamientos y compare las frecuencias relativas, la media y la varianza con los valores teóricos.

Ejercicio 3 (variable continua). Sea X con densidad $f(x) = cx^2$, $0 < x < 3$.

- Determine c y verifique con `integrate`.
- Calcule $P(1 < X < 2)$.
- Calcule $E(X)$ y $V(X)$.
- Obtenga la mediana y el percentil 90 con `uniroot`.
- Grafique F en el soporte usando `sapply`.

Ejercicio 4 (modelos discretos y desigualdades). En una línea de producción, el 8% de las piezas es defectuosa, independiente entre piezas. Se inspecciona una muestra de 25, y sea X el número de defectuosas.

- (a) $P(X = 0)$, $P(X \geq 3)$ y $P(2 < X < 5)$.
- (b) El percentil 95 de X ; verifique que cumple la definición.
- (c) Simule 10 000 muestras y compare la media y la varianza con np y $np(1 - p)$.
- (d) Ahora se inspeccionan piezas una a una. Sea N el número de piezas revisadas hasta encontrar la primera defectuosa (incluyéndola). Calcule $P(N \leq 10)$ y $E(N)$, cuidando la definición de geométrica que usa R.

Ejercicio 5 (modelo normal, teórico vs empírico). El peso de un paquete es $N(250, 12^2)$ gramos.

- (a) $P(X < 230)$ y $P(240 < X < 265)$.
- (b) Los valores a, b tales que $P(a < X < b) = 0.95$ con colas iguales.
- (c) Se rechazan los paquetes fuera de $[226, 274]$. ¿Qué proporción se rechaza?
- (d) Simule 10 000 pesos y superponga el histograma con la densidad teórica.

Parte VI – Soluciones

24 Soluciones

Los valores simulados dependen de la semilla y de la versión de R; lo importante es que queden cerca del teórico.

Ejercicio 1

```
choose(5, 2) * choose(7, 2) / choose(12, 4)    # 210/495 = 0.4242

ncol(combn(1:12, 4))      # 495
choose(12, 4)             # 495

x <- 0:4
p <- choose(5, x) * choose(7, 4 - x) / choose(12, 4)    # vectorial
p                                     # 0.0707 0.3535 0.4242 0.1414 0.0101
sum(p)                             # 1

sum(x * p)                         # E(X) = 1.667 = 4 * 5/12
1 - p[x == 0]                     # P(X >= 1) = 0.9293
```

`choose` acepta vectores igual que el resto de las funciones de R, así que con `x <- 0:4` se calcula la cuantía entera en una línea. La esperanza $1.667 = 4 \cdot \frac{5}{12}$ tiene sentido: cada una de las 4 extracciones es roja con probabilidad $5/12$, aunque no sean independientes.

Ejercicio 2

```
Omega <- expand.grid(dado1 = 1:6, dado2 = 1:6)
Y <- Omega$dado1 * Omega$dado2
sort(unique(Y))      # 18 valores: 1 2 3 4 5 6 8 9 10 12 15 16 18 20 24 25 30 36
fy <- table(Y) / nrow(Omega)
y <- as.numeric(names(fy))
p <- as.numeric(fy)
EY <- sum(y * p)      # 12.25
VY <- sum(y^2 * p) - EY^2    # 79.97
sum(p[y <= 6])       # 14/36 = 0.3889
Fy <- cumsum(p)
data.frame(y, p, Fy)
plot(y, Fy, type = "s", xlab = "y", ylab = "F(y)")

set.seed(70725)
n <- 10000
Ysim <- sample(1:6, size = n, replace = TRUE) * sample(1:6, size = n, replace = TRUE)
prop.table(table(Ysim))    # comparar con fy
mean(Ysim)
var(Ysim)    # aprox. 12.21 y 80.25
```

Chequeo: $E(Y) = 12.25 = 3.5^2 = E(\text{dado}_1)E(\text{dado}_2)$, porque los dados son independientes. El recorrido tiene huecos (no aparecen 7, 11, 13, ...), así que `table` solo muestra los valores posibles y la escalera de F salta solo en ellos. $P(Y \leq 6)$: los pares (1,1..6), (2,1..3), (3,1..2), (4,1), (5,1), (6,1), que son 14 de 36.

Ejercicio 3

$\int_0^3 cx^2 dx = 9c = 1 \Rightarrow c = 1/9$. $F(x) = x^3/27$.

```
f <- function(x){ x^2 / 9 }
integrate(f, lower = 0, upper = 3)           # 1
integrate(f, lower = 1, upper = 2)$value     # 7/27 = 0.2593
EX <- integrate(function(x){ x * f(x) }, lower = 0, upper = 3)$value # 2.25
EX2 <- integrate(function(x){ x^2 * f(x) }, lower = 0, upper = 3)$value # 5.4
EX2 - EX^2                                   # 0.3375
FX <- function(x){ integrate(f, lower = 0, upper = x)$value }
uniroot(function(x){ FX(x) - 0.5 }, interval = c(0, 3))$root # 2.381
uniroot(function(x){ FX(x) - 0.9 }, interval = c(0, 3))$root # 2.896
xs <- seq(0, 3, by = 0.01)
plot(xs, sapply(xs, FX), type = "l", xlab = "x", ylab = "F(x)")
```

Exacto: mediana = $\sqrt[3]{13.5} \approx 2.381$, $P_{90} = \sqrt[3]{24.3} \approx 2.896$. La mediana (2.38) es mayor que la media (2.25) porque la densidad carga la masa a la derecha y deja una cola larga a la izquierda.

Ejercicio 4

$X \sim \text{Bin}(25, 0.08)$.

```
dbinom(0, size = 25, prob = 0.08)           # 0.1244
1 - pbinom(2, size = 25, prob = 0.08)       # P(X>=3) = 0.3232
pbinom(4, size = 25, prob = 0.08) - pbinom(2, size = 25, prob = 0.08) # P(3<=X<=4) =
0.2780
qbinom(0.95, size = 25, prob = 0.08)       # 4
pbinom(4, size = 25, prob = 0.08)
pbinom(3, size = 25, prob = 0.08) # 0.9549 >= 0.95 y 0.8649 < 0.95

set.seed(70725)
Xs <- rbinom(10000, size = 25, prob = 0.08)
mean(Xs)
var(Xs) # aprox. 2.00 y 1.89 ; teoricos 2 y 1.84

# (d) N = X_R + 1, donde X_R = fracasos antes del primer exito (lo que usa R)
pgeom(9, prob = 0.08) # P(N <= 10) = P(X_R <= 9) = 0.5656
1 / 0.08 # E(N) = 12.5 piezas
```

(a) $2 < X < 5$ con enteros son $\{3, 4\}$; equivale a `sum(dbinom(3:4, size = 25, prob = 0.08))`. (d) Si se usara `pgeom(10, prob = 0.08)` se estaría calculando $P(N \leq 11)$, un error de una unidad típico.

Ejercicio 5

sd = 12.

```
pnorm(230, mean = 250, sd = 12)             # 0.0478
pnorm(265, mean = 250, sd = 12) - pnorm(240, mean = 250, sd = 12) # 0.6920
qnorm(0.025, mean = 250, sd = 12)           # a = 226.48
qnorm(0.975, mean = 250, sd = 12)           # b = 273.52
1 - (pnorm(274, mean = 250, sd = 12) - pnorm(226, mean = 250, sd = 12)) # 0.0455
set.seed(70725)
X <- rnorm(10000, mean = 250, sd = 12)
hist(X, probability = TRUE, breaks = 40)
x <- seq(200, 300, by = 0.1)
lines(x, dnorm(x, mean = 250, sd = 12), lwd = 2)
```

(c) Los límites están a $\pm 2\sigma$ de la media, por eso se rechaza $\approx 4.55\%$ (la regla 95–4.5 de la normal).

Apéndice A – Para verificar los ejercicios propuestos de los labs

Solo los valores, para comprobar lo que hagas tú (calculados en R):

Lab	Pregunta	Valor
L2 a–b	3 monedas: $P(X \geq 2)$, E , V	0.5, 1.5, 0.75
L2 c–d	$c(1+x)$, $0 < x < 1$: c , E , V	$2/3$, $5/9 \approx 0.5556$, $13/162 \approx 0.0802$
L2 d	percentil 60	$-1 + \sqrt{2.8} \approx 0.6733$
L3 a	$\text{Bin}(30, 0.4)$: $P(X \geq 15)$, $P(8 \leq X \leq 16)$, P_{90}	0.1754, 0.9084, 15
L3 b	$\text{Poi}(6)$: $P(X \leq 4)$, $P(X > 8)$, k	0.2851, 0.1528, 10
L3 c	$\text{Exp}(0.2)$: $P(X > 8)$, P_{75}	0.2019, 6.931
L3 d	$N(100, 15^2)$: $P(80 < X < 120)$; a , b	0.8176; 70.60, 129.40
L3 e	$\text{Lognormal}(1.5, 0.6)$: $P(X > 5)$, P_{90}	0.4276, 9.669
L3 Gumbel*	$\alpha = 2, \beta = 1.5$: $P(X \leq 4)$, P_{95}	0.7683, 6.455

* Sección *para estudio personal* del Lab 3.